Coding Cup TecnNM 2021

02:43:54

Problemas | Ranking | Clarificaciones **(8)**

G. Karel lite

Puntos	63.09	**Límite de memoria**	32 MiB
Límite de tiempo (caso)	1s	**Límite de tiempo (total)**	1m0s
Entrada/Salida	Consola	**Tamaño límite de entrada (bytes)**	10 KiB

Descripción

Karel es un simulador de un robot que se utiliza para comenzar en el mundo de la programación. Puedes encontrarlo en https://omegaup.com/karel.js/

Karel lite pretende ser una versión reducida de Karel, se encuentra en un mundo cuadrado de 20 x 20. Siempre está orientado hacia alguno de los puntos cardinales, es decir, norte, sur, este, oeste. La programación de Karel incluye solamente cuatro instrucciones:

turnleft(); El robot gira a la izquierda.

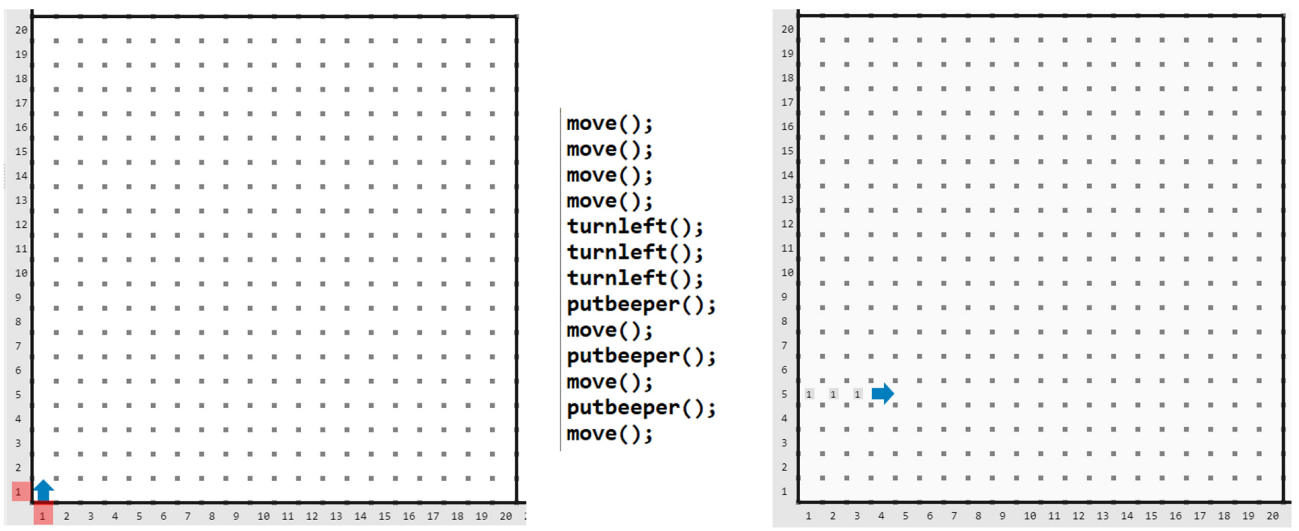
move(); Karel da un paso al frente hacia donde esté orientado.

putbeeper(); Coloca un beeper (señal) en el lugar donde se encuentra karel.

pickbeeper(); Toma un beeper del lugar donde se encuentra karel.

A partir de una posición y orientación inicial, la cantidad de beepers en la mochila y unas instrucciones de entrada, mostrar en que lugar terminará el robot, su orientación final y también debe imprimir como finalizó el mundo.

Ejemplo. Suponga que Karel inicia en la posición 1, 1 orientado al norte y con 10 beepers en la mochila, como lo muestra la imagen. Con las instrucciones que aparecen en el ejemplo, el mundo final quedaría como se muestra en la imagen.



Desarrolle un programa que lea la posición y orientación inicial de Karel e imprima el mundo final resultante de la ejecución del código.

A diferencia de Karel, en Karel lite siempre se inicia con el mundo vacío, es decir, sin beepers. En esta versión de Karel, no existen paredes intermedias que estorben el movimiento del robot.

Entrada

Primero se deben leer dos valores ***x*** y ***y*** correspondientes a la posición inicial de Karel.

En la segunda línea se encuentra la cantidad ***M*** de beepers que Karel trae en su mochila. Las tercer línea indica la orientación inicial de Karel y puede ser *North, South, East, West*.

la cuarta línea es un entero ***N*** que indica la cantidad de instrucciones del programa. Las siguientes ***N*** líneas incluyen el código que indica a Karel las instrucciones a seguir. Cada linea incluye solamente una instrucción. Es posible que alguna de esas instrucciones sea incorrecta, es decir, que esté mal escrita.

Salida

Si Karel choca con una pared, si intenta tomar un beeper donde no hay o si intenta poner un beeper y no trae beepers en la mochila, la salida será simplemente *"Run time error"*.

Si hay un error en las instrucciones porque se escribe alguna instrucción de manera incorrecta, por ejemplo *"turnright();"*, el programa debe imprimir solamente *"Sintax error"*. Otro error de sintaxis sería escribir una instruccion correcta pero sin ";", por ejemplo *"putbeeper()"*. En general, cualquier instrucción incorrecta llevaría a imprimir *"Sintax error"*.

Si no hay instrucciones incorretas ni errores de ejecución, el programa debe imprimir lo siguiente.

En la primer línea se indicará la posición final *(x, y)* de Karel.

En la segunda línea se escribirá la orientación final de Karel (North, South, East, West).

La tercer línea indica la cantidad final de beepers en la mochila del robot.

Posteriormente se imprimirá el mundo final de Karel. Este mundo de 20x20 estará representado por un punto (.) si la posición se encuentra vacía o por un entero que representa la cantidad de beepers en esa posición.

Ejemplo

Entrada	Salida

1 1
10
North
13
move();
move();
move();
turnleft();
turnleft();
turnleft();
putbeeper();
move();
putbeeper();
move();
putbeeper();
move();
	4 5
	East
	7
	....
	....
	....
	....
	....
	....
	....
	....
	....
	....
	....
	111.................
	....
	....
	....
	....

Límites

- $1 \leq x \leq 20$
- $1 \leq y \leq 20$
- $1 \leq N \leq 500$
- $0 \leq M \leq 99$

Fuente: undefined

code cases

Main.cpp

```cpp
#include <iostream>

int main() {
  std::cin.tie(nullptr);
  std::ios_base::sync_with_stdio(false);

  // TODO: fixme.

  return 0;
}
```

compiler.out/err logs.txt files.zip

Envíos

Enviado	GUID	Estatus	Porcentaje	Lenguaje	Memoria	Tiempo	Accio
Nuevo envío							

INCREMENTANDO EL NÚMERO DE INGENIEROS DE SOFTWARE TALENTOSOS EN AMÉRICA LATINA.

Patrocinadores

amazon

Sitio

Concursos

Cursos

Problemas

Ranking

Blog

Organización

Acerca de omegaUp

Equipo

Desarrolladores

Ayúdanos

Reporta un problema

Contacto

hello@omegaup.com

GitHub | Facebook | Discord

Copyright © 2021 omegaUp, Inc. Todos los derechos reservados.

Código de Conducta

Políticas de privacidad